m17web User Manual

M17 digital voice on a Raspberry Pi with an SDR, driven from your web browser

Version 1.2 — 10 November 2026

This manual is for anyone using the radio. It explains what the program does, how to start it, how to make a contact, and what to do when something does not work. You do not need to understand M17, SDR or Linux to use it — if someone set the system up for you, start at section 4.

A separate README in the same folder covers the technical details: building the program, the audio back ends, and the transmit timing.

What is in this manual

	Setting it up		Using it
1	What m17web does	7	Audio: where your voice comes from and goes
2	What you need	8	The controls, panel by panel
3	Starting and stopping it	9	Commands
4	Opening the interface	10	If something goes wrong
5	The screen at a glance, your position and the locator	11	Good practice
6	Making a contact, and the PTT switch	12	Quick reference

1. What m17web does

M17 is a digital voice mode for amateur radio: like FM voice, but it sends your speech as data, which gives clear audio and carries your callsign with it. m17web is a program that runs on a small computer (a Raspberry Pi) connected to a software-defined radio. It sends and receives M17 voice, and you control it entirely from a web page — no terminal commands needed in normal use.

Your voice comes from a microphone plugged into the Pi, or from the computer you are browsing from. Received voice comes out of the Pi's speaker (or your computer's).

If you are only practising, the program can run without a radio, in a mode that feeds your own audio straight back. See section 9.

2. What you need

Item	Notes
Raspberry Pi 4	Any 4 GB or 8 GB model, with a network connection. The Pi runs m17web.
ADALM-Pluto (PlutoSDR / Pluto+)	The radio itself, connected to the Pi by USB. A LimeSDR also works.
USB sound card with microphone and speaker or headphones	For audio at the Pi. Recommended - it is the most reliable audio path.
Antenna and a valid amateur licence	You need a licence for any frequency you transmit on.
A web browser	On any computer, tablet or phone on the same network as the Pi.
Optional: a GPS receiver	A small GPS receiver plugged into the Pi shows your position and Maidenhead locator on screen, and sends it to other stations (section 5).
Optional: a PTT switch	A switch wired to the Pi's header lets you transmit without the browser, from a hand microphone or a foot switch (section 6).

3. Starting and stopping it

m17web is started from a terminal on the Pi. If someone else set the system up for you, it may already be running — in that case go straight to section 4.

Start it (this also works after a reboot):

```
cd ~/m17web-src/m17gismo
RADIO=pluto MODE=radio bash tools/pi_m17web.sh start -a alsa
```

It keeps running after you close the terminal window. **Check it:**

```
bash tools/pi_m17web.sh status
```

Stop it, or look at the log to see what it is doing:

```
bash tools/pi_m17web.sh stop
bash tools/pi_m17web.sh log
```

What the settings mean

Setting	What it does
RADIO=pluto	Use the PlutoSDR. Use **RADIO=sim** instead to practise with no radio attached.
MODE=radio	Radio in, local sound out — the normal voice mode. It can also be changed while the program is running, from the Mode panel (section 8).
-a alsa	Send the voice to the Pi's sound card (see section 7). Use **-a browser** to use the browser's own microphone and speakers instead.

4. Opening the interface

In a browser on the same network, go to the address of the Pi followed by the port number:

```
http://10.168.1.33:8080/
```

Substitute the address of your own Pi. Bookmark it: this is the whole interface.

At the top right there is an **online / offline** marker. If it says offline, the page has lost contact with the Pi and nothing you press will do anything — check that the program is running (section 3) and reload the page.

5. The screen at a glance

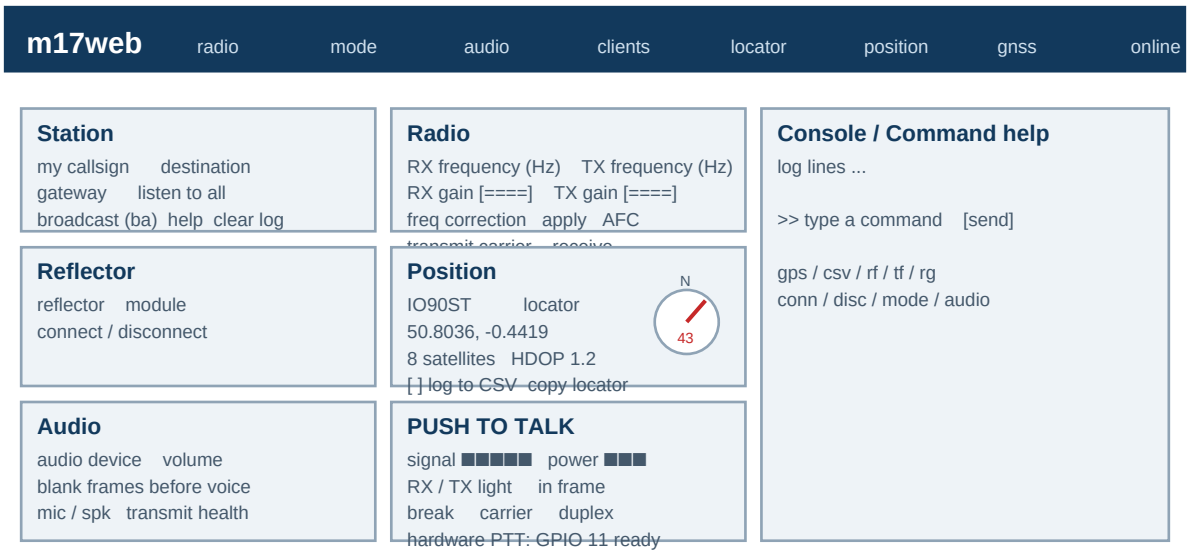

Figure 1: the panels on the page. The header is always visible and carries your locator and position. The cards on the left are laid out two across - Station and Radio, Reflector and Position, then Audio with PUSH TO TALK at the bottom right - and the console and command help are the column on the right. The cards are compact enough that the whole page fits a screen without scrolling; the signal and power bars sit at the top of the PUSH TO TALK card. (Sketch, not a screenshot.)

Area	What it is for
PUSH TO TALK	Bottom right. Press and hold while you speak, then release. You can hold the space bar instead. A physical switch on the Pi does the same job.
RX / TX light	Shows whether you are transmitting. Next to it, **in frame** means a valid M17 signal is being decoded, **no signal** means nothing is there. In client mode - where there is no radio and the audio comes from a reflector - the same box names the station you are hearing, as **hearing G4ABC**.
signal and power bars	The plain bars at the top of the PUSH TO TALK card: how strong the received signal is, and your transmit power setting. (The audio levels in the Audio card are the dials.)
Station	Your callsign, who you are calling, and the broadcast and log buttons.
Radio	The frequencies and gain settings, and the button that applies them.
Reflector	Linking over the internet to other M17 users (you may never need it).
Audio	Where your voice comes from and goes, the volume, and how healthy your transmit audio is.
Console	A command box for advanced settings, and the log that shows what is happening.
locator, position, gnss (top of the page)	Your Maidenhead locator and latitude/longitude from the GPS, and whether your position is being sent with your transmissions.

Your position: the locator, and the compass

If a GPS receiver is plugged into the Pi, your position is read from it and shown in two places: in the header at the top (locator, latitude and longitude) and in the **Position** panel, in large letters.

What you see	What it means
IO90ST	Your Maidenhead locator: six characters giving your position to about 5 km. Other operators use these to aim antennas, and they are the polite way to say where you are.
50.8036, -0.4419	Latitude and longitude in degrees. South and west are negative.
satellites / HDOP / altitude	How good the fix is. Six or more satellites and an HDOP below about 2 means a good one. Altitude is in metres.
copy locator	Puts the locator on the clipboard, ready to paste into a log or a message.
position ready / position sent / position off air	Whether your position is being sent with your transmissions. It is sent only while the fix is fresh, about every 5 seconds; it reads **position off air** when you have switched the sending off.
the compass	The bearing to the station you are hearing, with the distance in kilometres. The red needle points at them; the small yellow mark is the direction they say they are travelling. It needs a fix of your own to work.
heard G4ABC ...	The position another station sent, with its locator and how long ago it was received.

Your own position is sent to other stations as part of the M17 link setup, so send it only if you are happy for it to be public - it appears in the Location column of the reflector's own dashboard, and in any client that displays it. If you would rather not, untick **announce position on air** in the Position panel, or give the command **gps meta off**. The receiver keeps working either way, so the locator, the compass and the position log are unaffected, and the setting is remembered when you next start the program. Unplugging the receiver also works, if you want no position at all.

Keeping a log of your position

Tick **log to CSV** in the Position panel and a row is added to a file every few seconds, which you can open in a spreadsheet: date, time, latitude, longitude, locator, altitude, speed, course, satellites, HDOP, the age of the fix and a timestamp. The panel tells you where the file is — **gps_track.csv** in the program's folder on the Pi — and how many rows it holds. Untick it to stop. The **csv** command (section 9) does the same thing, and can change how often a row is written.

6. Making a contact

1. Type your callsign into **Station** → **my callsign**. It is sent with every transmission, so it must be correct.
2. Agree a frequency with the other station, then type it into **Radio** → **RX frequency (Hz)** and press **apply frequencies**. This sets transmit to the same frequency.
3. Listen first. When the other station speaks, the light shows **RX**, the signal bar moves, **in frame** appears and you hear them.
4. To reply, hold **PUSH TO TALK**, speak in your normal voice a little slower than a telephone call, then release.
5. Wait for the other station to finish before you press again — while you are transmitting, you cannot hear them.

Only transmit on frequencies your licence allows, and check what is already in use before you press. If you are testing, a dummy load or the lowest gain keeps you out of trouble.

Using a physical PTT switch

As well as the button on the page you can transmit with a switch wired to the Raspberry Pi's header — a hand microphone, a foot switch, or a small box you can hold. Pressing it transmits exactly as the button does, and the RX / TX light follows it, so you can put the browser down and operate normally.

Pin	What it is for
GPIO 11 (header pin 23)	One side of the switch. The other side goes to ground — for example header pin 25 or 9.
GPIO 10 (header pin 19)	An output that goes high while transmitting, to key an amplifier or another radio through a suitable interface.

Ask whoever set the system up before wiring anything to the header: the pins have to match, and the switch must go **to ground**, not to 3.3 V. Use a transistor or a relay for anything that draws real current — these are logic outputs, not a power supply. Note also that GPIO 10 and 11 are the SPI pins, so an SPI display would use them.

Releasing the switch returns you to receive. Releasing it does not cancel a transmission you started from the browser, so the two ways of transmitting do not fight each other.

Using the phone as the speaker and microphone

The Android app connects to the station as a client, and with the **speaker** and **microphone** both set to **phone** it becomes the station's audio: you hear the radio in your ear and speak into the phone, and the PTT button on the phone keys the transmitter. The page on the desk can stay open — it shows the meters, the log and every setting, and it does **not** take the audio.

Only a client that identifies itself as the phone gets the audio, so a page left open in another room does not compete with it. If the phone loses the network in the middle of a transmission the station releases the carrier by itself — immediately when the phone disconnects, or after three seconds without its audio — and says so in the log. The phone finds the station on the network by itself once the station advertises itself (see “tools/avahi” in the source); until then, its address is set once with **FN**.

This guard only applies when your voice is coming from a browser or a phone. With the sound card selected the microphone is local, nothing arrives over the network, and the guard stays out of the way. A transmission started with the physical switch is never cut short by it.

7. Audio — where your voice comes from and goes

The **Audio** panel has one button that matters: **Audio configuration…**. It opens a box with two sets of choices — one for the **speaker** and one for the **microphone** — and nothing changes until you press **Configure Audio**.

Choice	What it means	When to use it
browser	The microphone and speakers of the computer you are browsing from, carried over the network.	When you are away from the Pi, or it has no sound card.
phone	The **M17 app** on an Android phone: its speaker and its microphone, over the same network link. The phone also keys the station from its own PTT button.	A wireless microphone and speaker — the phone in your pocket while the page on the desk stays a control surface.
soundcard	The microphone and speaker plugged into the Pi's sound card, talked to directly.	**Recommended** when a card is attached: the most reliable choice.
bluetooth	A Bluetooth speaker or headset, chosen with **Configure Bluetooth**.	A tidy station with no wires. See below.
mute	Nothing: silence is transmitted, and nothing is played.	For testing, or to silence one direction only.

The two choices are independent, so the speaker can be a Bluetooth speaker while the microphone stays on the sound card, or the browser can carry the sound while the Pi's card takes the microphone. Press **Configure Audio** and the station detaches whatever that direction was using, attaches what you asked for, and keeps the choice for the next start. **(applying…)** means it takes effect at the start of the next transmit/receive cycle — never in the middle of a transmission.

If you ask for **bluetooth** before a device has been chosen, nothing moves: the station says to use Configure Bluetooth first, and that direction stays where it was. That is deliberate — a half-made request must not silence the station.

Configure Bluetooth

Configure Bluetooth opens the list of devices the station already knows. Each row has a **speaker** button and a **mic** button, there is a **scan** for new devices, and a **sound card** row at the top to put either direction back on the card.

Choosing a device here **remembers it and connects it, but does not move the audio**: press **Configure Audio** afterwards to attach it. That way a half-chosen request never takes the sound away.

The two buttons are independent, so the SoundCore mini can carry the sound (A2DP, playback only) while a headset carries the microphone (HFP, 8 kHz — exactly M17's own rate, so nothing is lost against Codec2). A headset that has no A2DP at all — the BH-M10B is one — shows **speaker (HFP)** instead of **speaker**, and its own earpiece carries the sound at 8 kHz; the station works that out from the device, so there is nothing to set. A device a scan has just found is only *discovered*: it shows **(not paired)** and a **pair** button instead, and nothing can connect it until it is paired.

If the device is switched off, or out of range, the station says so and keeps trying: it picks the device up by itself as soon as it is switched on, without a restart. With no sound card attached there is deliberately nothing to fall back to, so a Bluetooth-only station is a proper

arrangement, not a fault. Bluetooth audio needs the bluez-alsa packages on the Pi; without them the box says so and everything else is unchanged.

The two level controls, and the meters

The two meters are moving-coil instruments, not bars: a needle on a scale from -40 to +3 dB, the last three dB in red, a pale marker that holds the peak and falls back slowly, and a clip lamp in the corner that lights when the level reaches full scale (click it to reset). The needle is damped - quick to rise, slow to fall - so it is readable while you talk; the figure underneath is the exact level, and it turns amber above -6 dB and red at 0. When the audio stops - the other station finishes, or you release PUSH TO TALK - the needle falls back to rest within a second or two rather than hanging where the last frame left it.

Beside the meters are **mic level** and **volume**. Both are applied on the Pi itself: the microphone gain sits between the microphone and the modulator, and the volume between the receiver and the speaker. That is what makes the meters worth watching — the **mic** bar shows the level actually going out on air, not just what a browser happened to capture, and the **spk** bar shows what is being played.

The readouts are in decibels. **mic level** runs from 0 (silent) to 200% (+6 dB); 100% is 0 dB and suits most microphones, so raise it if you sound quiet on air and lower it if the bar is pinned while you speak normally. **volume** runs 0-100%, with 100% at 0 dB.

Both are kept for the next start, and the console does the same: **micgain 150** and **spkvol 60**.

Received audio is played through about a third of a second of buffer, which is what keeps the odd late moment from being heard as a gap. If the status line under the meters ever reports **speaker underruns**, the card itself ran dry — that number is worth quoting to whoever looks after the station, because no underruns at all means any break-up is happening before the audio gets to the card.

The card's own controls: the Mixer panel

A USB sound card has controls of its own — a volume, and often a microphone gain and an automatic-gain switch. These are **not** the same as the application's levels: they belong to the card, and one of them sitting in the wrong place is the usual reason the audio looks healthy while nothing can actually be heard. A C-Media headset, for instance, comes up at 29% (about -20 dB), which is quiet enough to lose received voice completely.

The **Mixer** panel lists whatever controls your card really has, and setting one takes effect at once. Whatever you leave them at is remembered and applied again every time the audio is opened — so a card that drops off USB and returns gets its levels back with it, and a reboot does not undo them. The console does the same: **mixer** lists the controls, and **mixer 0 100** sets one.

If the browser refuses your microphone

Browsers only allow microphone access on a secure page. A page opened as **http://<address>:8080/** is not secure, so the microphone may be blocked. Either open **http://localhost:8080/** in a browser on the Pi itself, or choose the Pi's sound card instead — which avoids the problem completely and is what most users want. The **enable browser audio** button beside the configuration button starts the browser's audio by hand if a saved

configuration needs it; Configure Audio does it for you otherwise.

Blank frames before voice, and the transmit health line

blank frames before voice is the amount of silence sent at the start of each transmission, before your speech. It gives the receiving station time to lock onto your signal, so your first word is not clipped. The default is 4 (about 80 thousandths of a second). If people tell you the start of your first word is missing, set it to 6 or 8.

The line beside it — **transmit clean**, or **dropped N samples** — is the health of your transmit audio. **Transmit clean** means the Pi kept up with the radio for the whole transmission. If it reports dropped samples, raise the blank frames a little and try again; if it keeps reporting drops, the Pi is too busy (close other programs) or something needs looking at by whoever maintains the system.

8. The controls, panel by panel

Station

Control	What it does
my callsign	Your callsign. Sent with every transmission.
destination	Who you are calling. Leave as ALL unless you are calling one specific station.
gateway	Used when linking through a reflector or repeater (advanced).
listen to all	Receive everything on the frequency, whoever it is addressed to.
broadcast (ba)	Address the transmission to everyone (the usual choice on a calling channel).
help / clear log	Lists every command in the log, or empties the log.

Mode

How the station is wired up. **Radio** is the normal voice mode. **Loopback** feeds your own audio straight back, for practice with no radio or aerial. **Gateway** and **client** link the radio, or your audio, to a reflector over the internet.

Setting	What it does
radio (normal)	Radio in, sound out. Use this for contacts on a frequency.
loopback (practice)	Your own voice comes straight back, so you can check the microphone and the settings without transmitting.
gateway	The radio is bridged to a reflector: what you hear on frequency goes out to the reflector, and what comes back from the reflector goes out on frequency.
client	Your voice goes to the reflector over the internet, with no radio involved.

Changing the mode takes effect at once — you do not have to stop and start the program. If you are transmitting when you change it, the transmission stops. The choice is kept, so the same mode is still selected after a restart.

Radio

The receive controls are stacked above the transmit ones.

Control	What it does
RX frequency (Hz)	The frequency you listen on, in hertz (145500000 = 145.5 MHz).
TX frequency (Hz)	The frequency you transmit on. Usually the same as RX.
RX gain / TX gain	Receive and transmit levels. Leave them as they are unless someone asks you to change them.
freq correction	Fine tuning, for when a station sounds slightly off. Change it in small steps.
apply frequencies	Press this after changing a frequency or a gain, or nothing changes.
transmit carrier	Sends a plain carrier with no voice - for tests and alignment only.
receive	Returns to receiving if a transmission is stuck on, or after a test.
noise	Practice mode only: adds noise to the simulated signal.

Reflector

Reflectors link M17 stations over the internet, so you can talk to people out of radio range. Type the reflector's name — they are written like **M17-M17** — or pick one from the list that appears as you type, choose the channel letter (the **module**), and press **connect**. Press **disconnect** when you have finished: it is good manners, and it frees the channel for others.

Watch the log while you do it. It either says which address it is connecting to, or it says that the name is not in the list — in which case check the spelling against the reflector's own web page. The badge next to the buttons shows whether you are linked.

Reflector addresses do change over time. If a reflector that used to work stops working, ask whoever maintains the system to refresh the list: the program reads it from a file on the Pi, and the file has to be brought up to date (there is a tool for it). A stale address looks exactly like a connect button that does nothing. If you only want local radio contacts, ignore this panel.

Position

Your GPS position and Maidenhead locator in large letters, the compass showing where the station you are hearing is, and the tick box that keeps a log of your position in a file. Section 5 explains what each part means.

Audio

Control	What it does
Audio configuration…	Where the speaker and the microphone each come from and go - see section 7.
enable browser audio	Starts this page's own audio by hand, if a saved configuration needs it and Configure Audio has not been pressed.
mic / spk meters	Two analogue dials: the microphone level going out and the speaker level being played, both measured on the Pi after its own level controls. The figure under each one is the exact level in dB.
mic level / volume	Microphone gain and speaker volume, applied on the Pi - see section 7.
mixer	The sound card's own volume and gain controls - see section 7.
blank frames before voice	Silence sent before your speech at the start of a transmission. Default 4.

Console

Every setting also has a typed command, and the console is where you type them. You do not need it for normal operation; the help button lists the commands, and section 9 lists them all. The log below it shows what the program is doing — useful when something does not work.

9. Commands

Typed into the Console box, or into the program's own terminal. Type one command per line.

Command	What it does
rf 145500000	Set the receive frequency, in hertz.
tf 145500000	Set the transmit frequency, in hertz.
rg 0.5 / tg 0.5	Set the receive or transmit gain, from 0.0 to 1.0.
fc 1.0001	Frequency correction, from 0.9 to 1.1.
afc on / off	Automatic frequency control.
tx / rx	Start or stop transmitting (the PTT button does the same).
td / tc	Transmit duplex, or transmit a plain carrier.
sa G4ABC	Source address: your callsign.
da ALL	Destination address: who you are calling.
ga G4ABC G	Gateway address.
ba	Broadcast: call everyone.
la on / off	Listen to all addresses, or only yours.
conn NAME C	Connect to a reflector, on module C.
disc	Disconnect from the reflector.
mode radio	Normal mode: radio in, sound out. Alternatives: **loop** (audio loops back for practice), **gate** (radio to reflector), **client** (sound to reflector). Takes effect at once, and is kept for the next start.
audio	Which route the speaker and the microphone are on, and the two levels.
audio apply spk bluetooth mic soundcard	What **Configure Audio** sends: attach these two routes. Each is **browser**, **soundcard**, **bluetooth** or **mute**. Kept in config.txt.
micgain 150	Microphone gain as a percentage of unity (100 = 0 dB, 200 = +6 dB), applied before the modulator.
spkvol 60	Speaker volume 0-100% (100 = 0 dB), applied to received audio.
audio alsa	Both directions at once, the old way: **browser**, **alsa**, **both**, **none**, or **pulse**.
mixer	List the sound card's own controls and what they are set to - see section 7.
mixer 0 100	Set control number 0 to 100%. **on** and **off** also work, for a switch. Kept in config.txt for the next start.
bt	List the Bluetooth devices the station knows, and which is in use - see section 7.
bt sel spk MAC	Choose and connect a speaker without moving the audio - what Configure Bluetooth does. **bt sel mic MAC** is the microphone; **off** clears it.
bt spk MAC	Choose a Bluetooth speaker (A2DP) *and* move the speaker output to it.
bt spk off	Put the speaker output back on the sound card.
bt mic MAC	Take the microphone from that Bluetooth headset (HFP).
bt mic off	Put the microphone back on the sound card.
bt scan	Look for Bluetooth devices for 20 seconds.
bt pair MAC	Pair and trust a device a scan found, before choosing it.

Command	What it does
bt off	Put the audio back on the Pi sound card.
lead 4	Blank codec frames sent before the voice - see section 7.
fargan	The neural receiver: whether FARGAN is built in, and whether it is on.
fargan on	Receive through FARGAN at 16 kHz instead of the classic decoder. Costs about half the Pi's CPU, and is remembered for the next start.
fargan off	Back to the classic decoder.
record	Is it recording, and where? Whatever you hear is what gets written.
record on	Record what the receiver decodes as a WAV in the station's working directory: 16 kHz when FARGAN is on, since that is what it synthesises, and 8 kHz with the classic decoder. Not remembered across a restart, and it stops itself after 30 minutes.
record off	Close the recording; the file is then complete.
gps	Show the GPS receiver, the fix and your locator. **gps /dev/ttyACM1 38400** uses a different receiver.
gps meta off	Stop sending your position to other stations; **gps meta on** starts again. The receiver keeps running, so the locator, the compass and the log are unaffected - see section 5.
csv on / csv off	Start or stop logging your position to a file - see section 5.
csv 10	Log a row every 10 seconds rather than every 5.
csv status	Show the log file and how many rows it holds.
load FILE	Run a list of commands from a file.
w	Clear the log pane.
b	Break: stop transmitting now.
q	Quit the program.
h	List the commands.

10. If something goes wrong

What you see	What to try
The page will not open at all	Check the address and the port (8080), and that the Pi is switched on and on the network. On the Pi, run **bash tools/pi_m17web.sh status**.
It says **offline** at the top	The page lost contact with the Pi. Reload the page. If it stays offline, check the program is still running on the Pi.
Nothing comes out of the speaker	Press **Audio configuration…** and check which route the speaker is on, that it is not **mute**, and that **volume** is up. On **soundcard**, check the speaker is plugged into the sound card (not the Pi's headphone socket) and look at the Mixer panel.
I chose bluetooth but the audio did not move	Configure Bluetooth only chooses and connects a device; press **Configure Audio** to attach it. If no device had been chosen, the station says so and leaves that direction where it was.

What you see	What to try
The other station says my audio breaks up, or starts late	Look at the transmit health line in the Audio panel. Raise **blank frames before voice** to 6-8 and try again. If it still reports dropped samples, the Pi is struggling with something else running.
I hear nothing when the other station speaks	Check you are on the same frequency, and that **in frame** appears when they talk. If the signal bar moves but there is no audio, try **afc on**, then nudge **freq correction** slightly.
The browser will not use my microphone	Browsers only allow the microphone on a secure page. Use **http://localhost:8080/** on the Pi itself, or route the microphone to **soundcard** or **bluetooth** in the Audio configuration box (section 7).
The light never goes to TX	If the header says offline, the page cannot reach the Pi. Otherwise something else may be using the radio — stop and start the program.
The sound card is reported as busy	Another program (or an earlier run) is using it. Stop other audio programs, then stop and restart m17web.
The program will not start (port already in use)	An older copy is still running: **bash tools/pi_m17web.sh stop**, wait a moment, then start it again.
I transmit but nobody hears me	Test your own audio first, with no second operator needed: set the **destination** to **PARROT** and transmit for a few seconds. The reflector echoes your voice straight back, so if you hear yourself, your station is fine and the problem is at the far end (a listener on the wrong module, or a browser that has not been allowed to play audio). Set the destination back to **ALL**, or press **broadcast (ba)**, when you have finished.
The reflector will not connect	Type the name exactly as the reflector publishes it (**M17-M17**, not the short form), press connect, and read the log: it says which address it is trying, or that the name is not in the list. If the list is out of date, ask whoever maintains the system to refresh it. Some reflectors only answer on particular modules.
The locator is blank, or there is no position	The GPS receiver has not got a fix yet, or is not plugged in. A fix can take a minute or two outdoors, and from inside a building it may never arrive. Nothing else needs it, so carry on.
The physical PTT switch does nothing	Check the wiring first (section 6). On the Pi, **gpioinfo** should list GPIO 10 and 11 as used by m17web. If it shows something else, or the panel reports no hardware PTT, ask whoever maintains the system.
The radio cannot be found at all	The Pluto re-appears as a USB network interface when it is plugged in, and it needs the address 192.168.2.2. Check it is connected, then ask whoever maintains the system — this is covered in the README.

11. Good practice

- Transmit only on frequencies your licence covers, and at the lowest power that works.
- Listen before you transmit, and give way to a station already in contact.
- Keep your callsign correct in the Station panel — it is how other operators know who is calling.
- When using a reflector, say where you are going and disconnect when you have finished.

- For tests, use a dummy load or a carrier into a low gain rather than transmitting into the antenna.
- If you leave the system running, mute the audio if you do not want your room heard on the air.

12. Quick reference

Start it

```
cd ~/m17web-src/m17gismo
RADIO=pluto MODE=radio bash tools/pi_m17web.sh start -a alsa
```

Open it

```
http://10.168.1.33:8080/ (your own Pi's address)
```

Make a contact

Step	Do this
1	Station → my callsign: type your callsign.
2	Radio → RX frequency (Hz): type the agreed frequency, press **apply frequencies**.
3	Listen until you hear the other station (**in frame** appears).
4	Press and hold **PUSH TO TALK** (bottom right), speak, then release.
5	Let the other station finish before pressing again.

The three things that put most people right

If	Then
You cannot hear anything	Check the frequency, and whether **in frame** appears when the other station speaks. Try **afc on**.
Nobody can hear you	Look at the transmit line in the Audio panel. If it reports dropped samples, raise **blank frames before voice** to 6-8.
The page will not reach the Pi	On the Pi: **bash tools/pi_m17web.sh status**, then reload the page.

Your locator and position are in the header at the top of the page; **log to CSV** in the Position panel keeps a track of where you have been. A switch wired to the Pi transmits just as the button does. Settings worth knowing: **audio device** (use the Pi's sound card, direct), **blank frames before voice** (default 4) and **volume** in the Audio panel; **help** in the Station panel lists every command.

The optional neural receiver is **Wojciech Kaczmarski SP5WWP**'s Codec2-mod-neural project, which synthesises the same Codec2 3200 frames with FARGAN - the vocoder from the Opus codec, by Xiph.Org Foundation - instead of the classic sinusoidal decoder. Codec2 itself is David Rowe VK5DGR's.

13. Where this came from, and further reading

m17web is a modification of **G4GUO/m17_core**, with the original terminal interface replaced by the web page described here. The M17 protocol it speaks is an open standard from the M17 Project; the voice coding is Codec2 by David Rowe and the M17 Project's Codec2-mod build. It uses the open-source libiio, ALSA, Codec2 and libgpiod libraries.

For the technical details — building it yourself, the audio back ends, the transmit timing and the rate planning — see **README.md** next to this manual, in the same folder.

The upstream project has no licence file, so this fork adds its own documentation only. Treat the software as you find it: it works, but check it against your own licence conditions before putting it on the air.

Notes

Frequencies in use locally, reflector names, who to call when it breaks: